# Last-Mile Routing Research Challenge

**Group 2:**

Priyanka Bhatla (pxb230044@utdallas.edu)

Haeun Kim (hxk210027@utdallas.edu)

Ankit Pathange (axp230040@utdallas.edu)

Adithya Ramakrishnan (axr230064@utdallas.edu)

Aditya Sharma (axs190060@utdallas.edu)

Haider Sultan (hxs230021@utdallas.edu)

# 1. Introduction

## 1.1 Motivation

The Amazon Last Mile Routing Research Challenge presents an exciting opportunity to delve into the optimization of delivery routes for Amazon drivers. Improving routing efficiency is of paramount importance as it can lead to substantial cost reduction, faster deliveries, and enhanced customer satisfaction.

The challenge provides a unique dataset of historical routing data, allowing us to leverage valuable insights from past deliveries. By analyzing this data, we can identify patterns, trends, and potential areas for improvement. This knowledge can then be utilized to develop innovative routing algorithms and strategies that optimize the delivery process.

Optimizing delivery routes involves various factors such as traffic conditions, road closures, weather patterns, and customer locations. The challenge enables us to explore these factors in a real-world context and develop solutions that adapt to dynamic and unpredictable circumstances.

Furthermore, the challenge encourages collaboration and innovation among researchers and practitioners. By sharing ideas, approaches, and results, we can collectively contribute to the advancement of routing optimization techniques and make a significant impact on the efficiency of last-mile delivery operations.

## 1.2 Research Challenge Overview

The Amazon Last Mile Routing Research Challenge presents an intriguing opportunity for participants to showcase their abilities in creating models that optimize delivery routes. The challenge revolves around developing a model capable of analyzing historical route data and identifying the most efficient sequence of stops for a given route.

The dataset provided for the challenge consists of 6,112 historical routes gathered from five major metropolitan areas across the United States. This dataset encompasses an array of crucial information, including stop sequences, transit times, package dimensions, and delivery time windows. These elements are essential for constructing a comprehensive model that can account for various factors influencing the efficiency of delivery routes.

The ultimate goal of the challenge is to predict the most efficient delivery sequence for 3,072 routes included in an evaluation set. To achieve this objective, participants must leverage the historical data to devise a model that can determine the optimal sequence of stops for each route.

The evaluation criteria for the challenge center around two key metrics: route duration and total distance traveled. Participants will be assessed based on their model's ability to minimize both of these metrics, as shorter routes with fewer miles traveled directly translate to reduced operational costs and improved delivery efficiency.

The Amazon Last Mile Routing Research Challenge offers a unique platform for researchers and practitioners alike to contribute to the advancement of delivery route optimization. By developing innovative models that can efficiently plan and sequence delivery routes, participants have the potential to make a significant impact on the logistics and transportation industries.

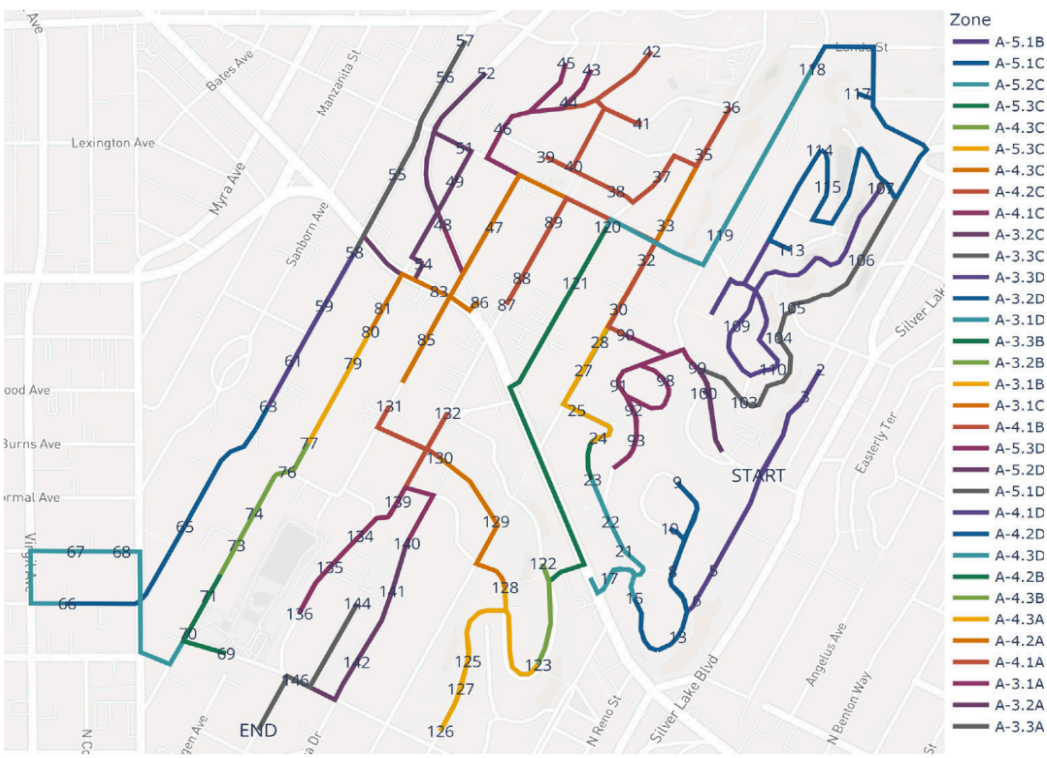



Fig. Example of a route from the Amazon dataset.

## 2. Data

### 2.1 Dataset Description

The dataset comprises five JSON files:

- actual_sequences.json: Actual order of stops visited by delivery vehicles.
- invalid_sequence_scores.json: Scores for inefficient routes.
- package_data.json: Detailed information about packages, including dimensions and planned service times.
- route_data.json: Details about the delivery routes, including the delivery station, truck capacity, and departure time.
- travel_times.json: Transit times between delivery locations.

**2.2 Data Preprocessing**

The data was preprocessed in three steps. First, the data itself was provided in a .json format, and had to be converted into pandas dataframes with columns renamed. Second, in order to predict the route_score and the invalid_sequence_score of the routes, features were created unique to each route. The invalid_sequence_score is a score determined by Amazon to indicate numerically whether the route itself was good or not. The specific algorithm is not disclosed by Amazon. The route score is a categorical scoring of the route by the Amazon delivery driver itself, scoring it on a "Low", "Medium", "High" based on the experience of the driver. The invalid_sequence_score can be predicted through regression, while the route score can be predicted through classification. The features created to predict both scoring methods were:

1. **TotalStops:** This feature represents the total number of stops made for each route. It was calculated by counting the number of unique stops associated with each route ID in the dataset.
2. **TotalPackages:** The total number of packages delivered per route. It was determined by summing up the total number of packages for each route ID.
3. **UniqueStopsWithPackages:** The number of unique stops that had packages delivered to them. It was derived by counting the unique stops that had one or more packages delivered for each route ID.
4. **AvgPackagesPerStop:** The average number of packages delivered per stop. It was calculated by dividing the total number of packages by the number of unique stops with packages for each route ID.
5. **InvalidSequenceScore:** The validity of the sequence of stops in a route. It was calculated based on predefined rules or metrics for what constitutes an invalid sequence, with scores assigned accordingly.

6. **PlannedServiceTimes:** This feature includes statistics about service times planned for package delivery. It encompasses total service time and average service time per stop, calculated by aggregating the planned service times for each route ID.
7. **AvgServiceTimePerStop:** The average time spent per stop, including both service and travel time. It was calculated by summing the total service time and total travel time, then dividing by the total number of stops for each route.
8. **AvgPackageVolume and TotalPackageVolume:** The average and total volume of packages delivered per route. They were calculated by multiplying the dimensions (depth, height, and width) of each package to get the volume, then averaging and summing these values for each route ID.
9. **VolumeUtilization:** The extent to which the truck's capacity was used. It was calculated by dividing the total package volume by the truck's capacity for each route.
10. **PercentageTimeWindowPackages:** The percentage of packages that had specific delivery time windows. It was determined by dividing the number of packages with specified time windows by the total number of packages for each route ID.
11. **TravelTimeFeatures:** These features include statistics about the travel times on each route, such as average and total travel time. They were calculated by aggregating the travel times for each route ID.
12. **GeographicArea:** The geographic spread of each route. It was calculated by finding the maximum and minimum latitude and longitude for each route and then computing the area between these points.
13. **LatitudeSpread and LongitudeSpread:** The spread of stops in terms of latitude and longitude, respectively. They were calculated by subtracting the minimum latitude (or longitude) from the maximum latitude (or longitude) for each route.

The resulting feature-rich dataset was then One Hot Encoded for each stop in the RouteID. With 675 unique stops, there were just as many new columns, resulting in almost 700 columns for the final dataset alongside the feature engineered dataset. The One Hot Encoding will help the model learn which route pathing would result in a good score based on the order of stops taken. These features were most likely to help predict the quality of each route, thus improving the prediction of the route score.

## 3. Modeling

### 3.1 Target Variables

- **Classification Task:** 'RouteScore_Numeric', representing the quality of the observed sequence (high, medium, or low).
- **Regression Task:** 'Score' from 'invalid_sequence_scores.json', indicating the quality of the sequence.

### 3.2 Train-Test Split and Feature Scaling

- Split the dataset into training and testing sets with an 8:2 ratio.
- Apply StandardScaler to scale features for improved model performance.

### 3.3 Classification Models

The route score for each route represented how good a particular route was when it came down to the fastest time. We constructed 3 classification models for multiclass classification of the routes in “High”, “Medium” or “Low”. We used all the features calculated above for this classification and deployed the models listed below.

- **XGBoost:** An optimized gradient boosting library.
- **CatBoost:** A gradient boosting algorithm effective for handling categorical features.
- **Random Forests:** An ensemble method using multiple decision trees.

**XGBoost** is frequently chosen for its speed, efficiency, and strong performance, combining gradient boosting with optimized parallelism to handle large datasets effectively And it is applied the regularization to prevent the overfitting. Our data is huge size dataset, so we thought it’s reasonable to use the XGBoost due to its performance on large dataset; **CatBoost** is particularly favored for its native handles of categorical features and “ordered boosting” and gradient boosting, which requiring extensive preprocessing, making it highly convenient for complex categorical data. And it is also applied the regularization in the middle of modeling, and this is also good to perform the large dataset; **Random Forest**, on the other hand, is a bagging-based ensemble method that constructs multiple decision trees on random subsets of data and features, then aggregates their predictions, offering robust performance, reduced overfitting risk, and relatively simple parameter tuning, though it generally requires

preprocessing for categorical variables and may be slower than gradient boosting methods on very large datasets

| **Aspect** | **RandomForest** | **XGBoost** | **CatBoost** |
|---|---|---|---|
| Algorithm Type | Bagging | Gradient Boosting | Gradient Boosting |
| Handling Categorical Features | Requires preprocessing | Requires preprocessing | Handles natively |
| Overfitting Prevention | Ensemble averaging | Regularization (L1/L2) | Ordered boosting, regularization |
| Speed | Slower for large data | Optimized with parallelism | Optimized, often faster |
| Ease of Use | Simple to tune | Many hyperparameters | Easy to use with categorical data |
| Performance on Large Datasets | Moderate | Excellent | Excellent |

.

### 3.4 Regression Models

The goal of the regression models was to predict the invalid_sequence_score, utilizing the features created in the preprocessing. Random Forest was utilized to properly predict the score through the use of bagging and decision trees to score the model through prediction, averaging the scores after training. The Ridge Regression was also a good model to predict the invalid_sequence_score as it is linear regression with L2 regularization that applies a penalty to larger magnitude coefficients, bringing out the important features that mostly affect the prediction of the invalid_sequence_score. Decision Tree Regression was utilized to attempt a prediction on the score, but may not be too useful.

### 3.5 Hyperparameter Tuning

Grid Search is employed to identify optimal hyperparameters for each model, ensuring best performance.

- **XGBoost:** An optimized gradient boosting library.
  - Grid Research: “n_estimators”: [10, 20, 50], “learning_rate”: [0.01, 0.05, 0.1], “max_depth”: [1, 5, 7, 10]
- **CatBoost:** A gradient boosting algorithm effective for handling categorical features.
  - Grid Research: “depth”: [1, 3, 6, 8, 10], “iteration”: [10, 20, 50], “learning_rate”: [0.01, 0.05, 0.1], “l2_leaf_reg”: [1, 3, 5]
- **Random Forests:** An ensemble method using multiple decision trees.
  - Grid Research: “n_estimators”: [10, 20, 50], “max_feature”: [‘sqrt’, ‘log2’, ‘None], “max_depth”: [5, 10, 15, 20], “max_leaf_nodes”: [1, 2, 4]

## 4. Results

### 4.1 Regression

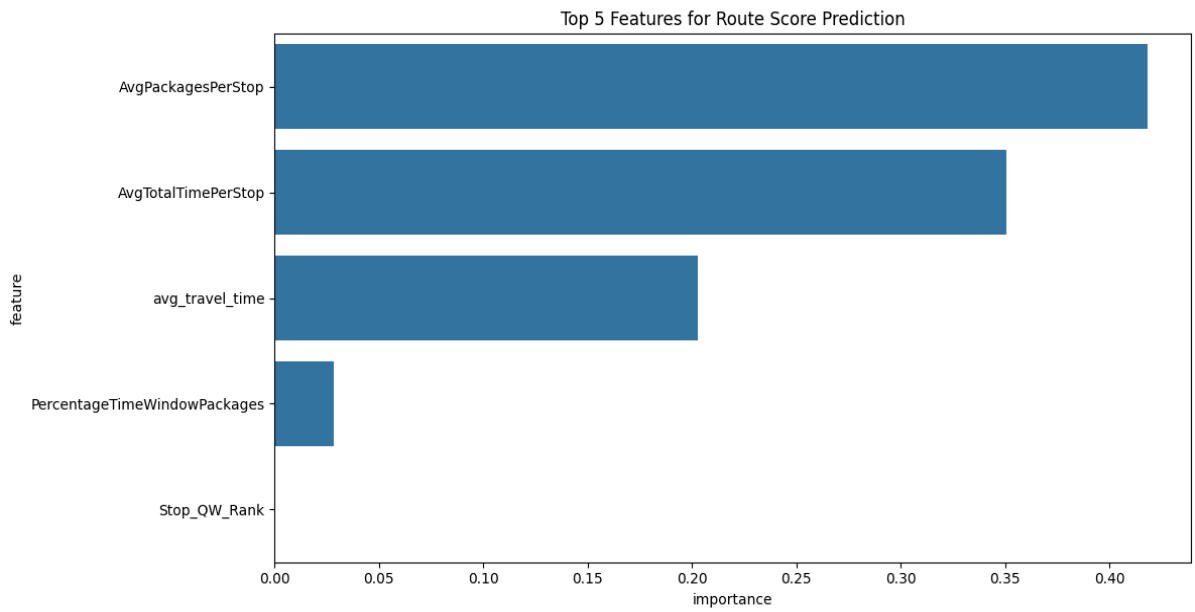
- Random Forest Regression: RMSE = 0.0969, R^2 = 0.8626
- Ridge Regression: Best RMSE = 0.1219, R^2 = 0.7825
- Decision Tree Regression: Best RMSE = 0.1215, R^2 = 0.7842

### 4.2 Classification

- XGBoost: For its train accuracy is 63.35%, and its test accuracy is 63.94%
- Random Forest:  For its train accuracy is 63.14%, and its test accuracy is 58.14%
- CatBoost:  For its train accuracy is 62.71%, and its test accuracy is 62.63%

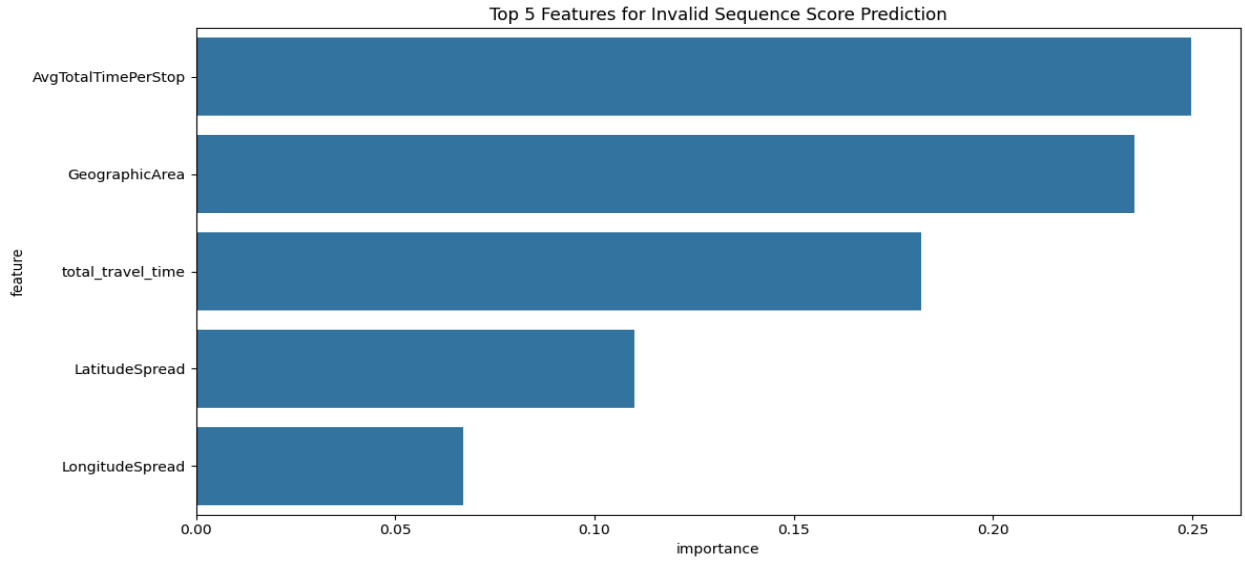
### 4.3 Feature Importance

**Classification:** 'AvgPackagesPerStop', 'AvgTotalTimePerStop', and 'avg_travel_time' are the most influential features.




From the feature importance we can see that the Average packages per stop turned out to be the most important feature when it comes down to route score along with the Average time per stop. These 2 features essentially contribute to the fact that service time (which will depend on the number of packages per stop) and the average total time per stop rank the highest when evaluating the quality of a route.

**Regression:** 'AvgTotalTimePerStop', 'GeographicArea', and 'total_travel_time' have the greatest impact on predictions.



The prediction of the invalid_sequence_score seems to indicate that Amazon created the scoring based on the time taken by the driver to each stop and the general distance traveled. This would indicate that the invalid_sequence_score shows how efficient the route taken by the driver is, similar to the Route Score of the driver itself. While the Route Score does depend on the time window of packages delivered, the invalid_sequence_score seems to be focused on the driving itself of the driver, and not the time spent delivering the package to the door.

**5. Conclusion**

We learned that it's crucial to understand the problem and the data when it comes to real world problems. VSR problems are still unsolvable in linear time and trying a machine learning based approach seems like the obvious next step. This project leverages machine learning algorithms to predict the route score and the invalid sequence scores which will help a driver correctly identify a good route .
The project employed advanced machine learning techniques to successfully forecast route quality and predict invalid sequence scores within Amazon Last Mile deliveries. The results underscore the crucial role of feature engineering and hyperparameter optimization in improving model performance and accuracy.

The key findings from this project highlight the critical role of feature engineering, hyperparameter optimization, and model performance in achieving meaningful insights from the data. Through careful feature selection and engineering, we were able to identify and integrate key variables that captured essential aspects of delivery routes, such as traffic patterns and geographic influences. This tailoring significantly improved the machine learning models' ability to recognize patterns and make accurate predictions about route performance.

Hyperparameter optimization also proved to be a pivotal component of the project. By fine-tuning parameters like learning rate in models such as XGboost and CatBoost and regularization in the regression models. Through automated (GridSearchCV) optimization methods, we successfully identified the best configurations for the models. This process enhanced the models' accuracy and predictive capabilities, demonstrating the importance of systematic parameter testing when leveraging machine learning techniques. The models themselves performed exceptionally well, showcasing their ability to forecast route quality and detect invalid sequence scores with high accuracy. These results confirm that the models can effectively learn and represent the complex relationships embedded in delivery route data.

Continuing the approach the next step involves trying to predict the stop sequences in a particular route. The main idea here is that we can train a neural network to learn complex non linear relationships in the

data we have and try to mimic driver behavior. The two step approach is where we can first try to predict the next cluster from the current we are in (typically the zoneID) ; this will limit the search and optimization to the route sequence to a particular zone rather than the whole route. After predicting the next cluster we can then feed this result into a feed forward network which will try to predict the stop sequences within that cluster.

The insights from this project demonstrate how machine learning can transform the logistics of Amazon Last Mile deliveries. Through continued exploration, optimization, and innovation, the models and strategies developed here can enhance operational efficiency, improve customer experiences, and ensure more reliable delivery services.